🧠 BCI-Controlled Toy Wheelchair - Complete System

A toy-scale prototype of a Brain-Computer Interface (BCI) controlled wheelchair for college demonstration. This is the **complete system** including EEG dataset download, preprocessing, SVM training, and real-time control.

📁 Project Structure

```
bci_wheelchair/
├── BCI_Wheelchair_ESP32_UNIVERSAL.ino   # ESP32 firmware (already flashed!)
├── laptop_controller_svm.py           # FINAL controller (SVM + Keyboard + Manual)
├── download_dataset.py                  # Auto-downloads PhysioNet EEG dataset
├── eeg_preprocessing.py                 # Filters EEG, extracts CSP features
├── train_svm.py                         # Trains SVM classifier
├── requirements.txt                     # Python dependencies
├── README.md                            # This file
└── models/                              # Created by scripts
    ├── features.npz                     # CSP features + labels
    ├── svm_model.pkl                    # Trained SVM model
    ├── csp_transformer.pkl              # CSP feature extractor
    ├── scaler.pkl                       # Feature normalizer
    └── class_mapping.pkl                # Class name mappings
```

🚀 Quick Start (Do This In Order)

Step 0: Install Python Dependencies

```
pip install -r requirements.txt
```

Step 1: Download EEG Dataset (Run & Go to Sleep!)

```
python download_dataset.py
```

This downloads the **PhysioNet EEG Motor Movement/Imagery Dataset** (10 subjects, ~60 EDF files, ~300-500 MB total). It has:

- 109 volunteers (we use 10 for the prototype)
- 64 EEG channels at 160 Hz
- Motor imagery tasks: left fist, right fist, both fists, both feet
- Mapped to wheelchair commands: **left, right, forward, backward**

Step 2: Preprocess EEG & Extract Features

```
python eeg_preprocessing.py
```

This will:

1. Load all downloaded `.edf` files
2. Bandpass filter (8-30 Hz — mu & beta rhythms for motor imagery)
3. Extract epochs around motor cues
4. Compute **Common Spatial Pattern (CSP)** features
5. Save processed features to `models/features.npz`

Step 3: Train SVM Classifier

```
python train_svm.py
```

This trains an SVM with RBF kernel on the CSP features and saves:

- `models/svm_model.pkl` — the trained classifier
- `models/training_report.txt` — accuracy report

Step 4: Update ESP32 IP Address

Edit `laptop_controller_svm.py` and set:

```
ESP32_IP = "192.168.1.100"   # <-- Your ESP32's actual IP from Serial Monitor
```

Step 5: Run the Controller!

Option A: SVM Mode (default)

```
python laptop_controller_svm.py
```

Uses the trained SVM to classify EEG and send commands to ESP32.

Option B: Keyboard Control (for testing/demo)

```
python laptop_controller_svm.py --keyboard
```

Control the wheelchair with arrow keys on your laptop!

Option C: Manual HTTP Test

```
python laptop_controller_svm.py --manual
```

Type commands interactively to test ESP32 connectivity.

🎮 Keyboard Controls (for –keyboard mode)

Key	Command
↑ UP arrow	forward
↓ DOWN arrow	backward
← LEFT arrow	left
→ RIGHT arrow	right
SPACE	stop

Generated by Kimi

Table 1 – continued

Key	Command
Q or ESC	quit

🧪 How to Test Without EEG Hardware

Since you don't have an EEG headset yet, here's the workflow:

1. **Use `--keyboard` mode** to drive the car and verify everything works
2. **The SVM model is trained** on the public dataset and will work with simulated data
3. **For your actual demo**, replace `simulate_eeg()` in `SVMClassifier` with real EEG acquisition code (see comments in the code)

🔌 Hardware Reminders

- **ESP32**: Already flashed with `BCI_Wheelchair_ESP32_UNIVERSAL.ino`
- **HC-SR04 Echo pin**: Must go through voltage divider (5V → 3.3V) before GPIO 18
- **Potentiometer**: Wire between GND and 3.3V (NOT 5V!)
- **Motor power**: Use separate battery pack, not USB power

🛠️ Troubleshooting

Problem	Solution
`mne` not found	`pip install mne`
`keyboard` module requires root	Run with `sudo` or use `--manual` mode instead
ESP32 not reachable	Check IP address, ensure same WiFi network
SVM model not found	Run steps 1-3 first (download → preprocess → train)
Download is slow	Normal —PhysioNet servers can be slow. Let it run overnight.

📊 Dataset Mapping

The PhysioNet EEGMMIDB dataset maps motor imagery tasks to wheelchair commands:

EEG Task	Wheelchair Command	Class Label
Left fist imagery	Turn left	`left`
Right fist imagery	Turn right	`right`
Both fists imagery	Move forward	`forward`
Both feet imagery	Move backward	`backward`

Table 3 – continued

EEG Task	Wheelchair Command	Class Label
Rest / no task	Stop	`stop` (inferred)

🎓 For Your College Demo

Recommended demo flow:

1. Show the ESP32 web server working via browser
2. Drive the car with `--keyboard` mode (arrow keys)
3. Show obstacle override with hand in front of ultrasonic sensor
4. Show potentiometer fallback (turn knob, wait 3 seconds)
5. Mention the SVM is trained and ready for real EEG integration

---

Good luck with your showcase!